

A PRIMER TO SORTING ALGORITHMS

Danilo Roccatano

School of Mathematics and Physics, University of Lincoln, Lincoln, UK.

INTRODUCTION

We live in a world characterized by an enormous amount of digital information; to find it, we need to put it in order. Just think about libraries with hundreds of thousands of books to place in the right bookshelves; the tax offices track every pound we earn; the credit bureaus that need to keep updated on almost all actions of their clients. The problem of keeping the information to retrieve it quickly is undoubtedly one of the significant problems in the information technology field. Because of the importance of keep data in order, several algorithms have been invented, the so-called sorting algorithms. There are so many different and essential ideas among all sorting techniques that a complete course could be taught on the subject. This primer aims to give an overview of the topics presenting some of the most common algorithm and their implementation in C++ language.

These methods fall into two primary groups: internal and external sorts. If the information to be sorted can be kept in a central, high-speed memory, the latter if this is not possible, i.e., if the information is so much that it must be kept externally on tapes, disks, cards, strips, etc. In the following, we will consider only types of internal arrangements. We will consider five methods selected based on their simplicity of implementation, efficiency for specific applications, and the difference in approach. Since the aim is to illustrate different sorting techniques, in the following, we will only consider the problem of placing numbers that are initially arranged in a list of N places in ascending order. This list will then be the input to our algorithms, and the same list, but with the numbers in ascending order, will be the output. Sorting algorithms must do two basic things: compare different values and swap them. The time taken in these two actions will influence the time of the algorithm. Therefore, the analysis will be based on the amount of these two operations.

SORTING BY INSERTION

Seasonal holidays just finished, and many of you have probably enjoyed playing card games. If a player has a deck of play cards, she almost automatically learns to sort them out in his hand. Usually, she will look at one card at a time and put each new card in its place. This method can be easily implemented as an algorithm for calculators. To understand how this algorithm works, assume that the cards are laid face up on a table. To sort them, we look at them from left to right, one at a time. When we see a new card, we compare it to the cards seen before, and if the value is less than theirs, it is moved back to one position to insert the new one. In Figure 1, the procedure is schematically shown.

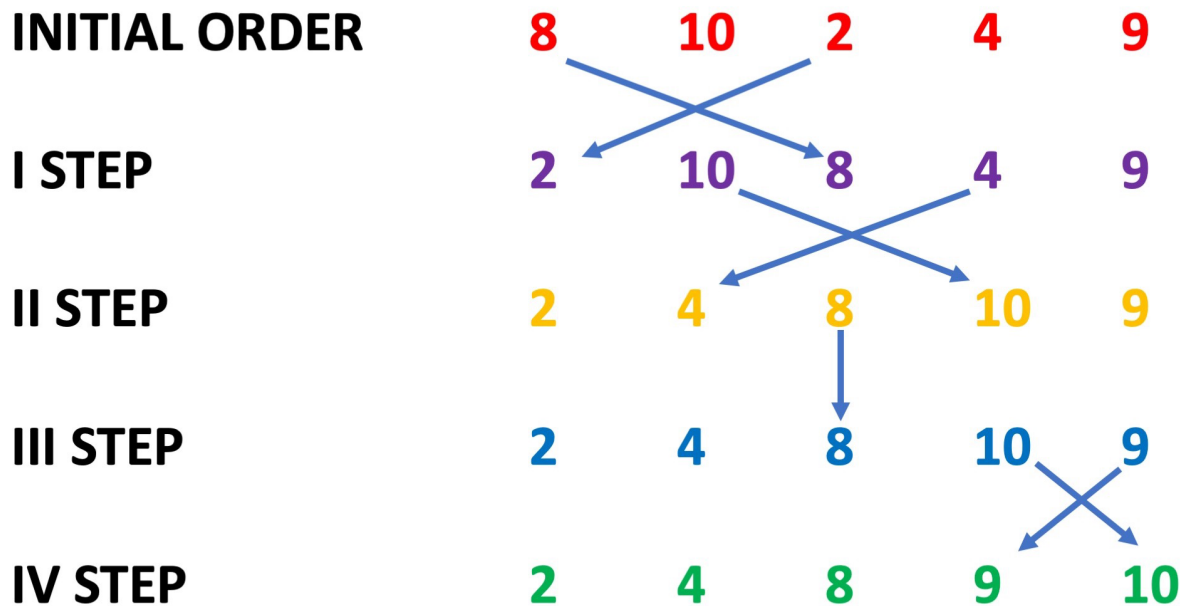


Figure 1: This example shows sorting a set of cards using the insertion method. The arrow in each step indicates the exchanges between pairs of cards.

The sorting by insertion algorithm is based on keeping in order part of the list that has already been examined. In this way, a list with only one argument is automatically sorted. However, if we have sorted the first $I-1$ elements, we take the element I^{th} and look in the ordered part of the list to find where we can insert it.

A little trick to simplify the algorithm is to use an already sorted list from the bottom, so we can search and simultaneously move the arguments to insert the element I when we find its place. In the following C++ program, an example of the implementation of the insertion algorithm is given.

```
#include<iostream>
using namespace std;

#define MAX 100

int main() {
    int n, list[MAX], i, j, temp;

    cout<<"how many element?: ";
    cin>>n;

    /*
    Input n elements (max number is given by MAX value
    */
    for(i=0;i<n;i++) {
        cout<<"Element number: "<<i+1<<" ";
        cin>>list[i];
    }

    cout<<"Sorting the array:\n";
    for(i=1;i<n;i++) {
        temp=list[i];
        j=i-1;
        while((list[j]>temp) && (j>=0)) {
```

```

        list[j+1]=list[j];
        j--;
    }
    list[j+1]=temp;
}

cout<<"Array after sorting: \n";
for(i=0; i<n; i++)
    cout<<list[i]<<endl;

return 0;
}

```

This algorithm is efficient if the list consists of already largely sorted elements. However, on the other hand, if many elements are out of order, the algorithm is very slow, especially if the elements are out of order at the list's end. The behaviour of this algorithm is, therefore, not well balanced. The theoretical analysis makes it possible to determine the number of comparisons and shifts to be carried out to order N .

	Minimum	Average	Maximum
Comparisons	$N - 1$	$\frac{N^2 + N - 2}{4}$	$\frac{N^2 + N}{2} - 1$
Displacements	$2(N - 1)$	$\frac{N^2 - 9N - 10}{4}$	$\frac{N^2 + 3N - 4}{2}$

SORTING BY SELECTION

The sorting by insertion method has one major drawback: even when almost all items have been sorted in the first part of the list, inserting a new element can cause many elements to move. That is because all moves in an insertion sort are moved one position at a time, so to move an element of, for example, ten places require ten separate moves.

If the elements move are just numbers as they occupy only a few bytes of memory, these movements are not so slow. However, if the elements are significant, for example, records of hundreds of bytes, it would be better when it is necessary to move an element if it could be immediately moved to its final position. The sorting-by-selection method takes this problem into account.

We can again imagine a player who likes to see all his cards simultaneously. She chooses the lowest and puts it first when he looks at them. Then, he chooses the second lowest and puts it second; keep doing this until all the cards are in order. In Figure 2, the procedure is schematically shown.

INITIAL ORDER

I STEP

II STEP

III STEP

IV STEP

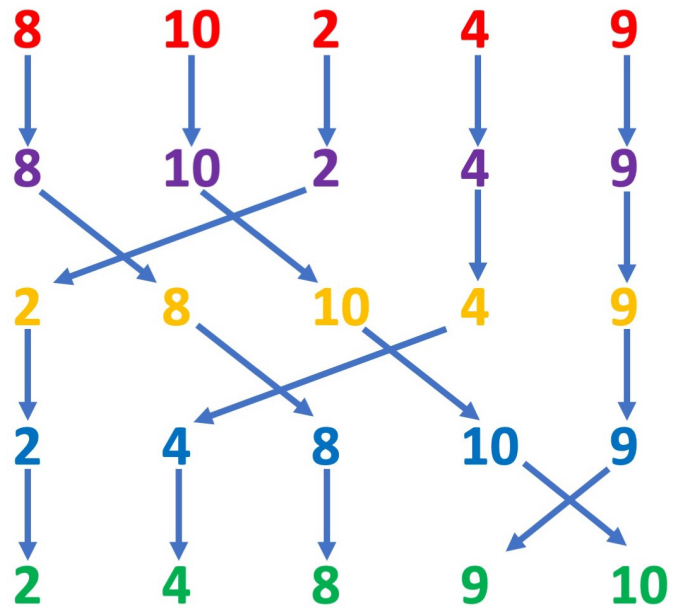


Figure 2: This example shows sorting a set of cards using the selection method. The arrow in each step indicates the exchanges between pairs of cards. In step 1, the minimum value (2) is first searched.

In the following C++ program, an example of the implementation of the selection algorithm is given.

```
#include<iostream>
using namespace std;

#define MAX 100

int main()
{
    int n, i, j, min, temp;
    int list[MAX];

    cout<<"how many element?: ";
    cin>>n;

    /*
    Input n elements (max number is given by MAX value
    */
    for(i=0;i<n;i++) {
        cout<<"Element number: "<<i+1<<": ";
        cin>>list[i];
    }

    cout<<"Sorting the array...\n";

    for(i=0;i<n-1;i++)
    {
        min=i;
        for(j=i+1;j<n;j++)
            if (list[j]<list[min])
                min= j;

        temp=list[min];
        list[min]=list[i];
        list[i]=temp;
    }
}
```

```
cout<<"Array after selection sorting: \n";
for(i=0;i<n;i++)
cout<<" "<<list[i] << endl;

return 0;
}
```

We observe that the aim is to minimize the number of displacements of the elements. That is done by identifying the element with the smallest values first. Once it is found, a switch between the current and minimum value elements is made. The search for the smallest element is carried out on the part of the list not yet sorted. Note that if all but one of the elements is in order, the remaining elements must also be. Therefore, the for-loop loop will stop at N-1 since the last argument is always the maximum.

As far as the algorithm's analysis is concerned, we observe that the number of comparisons made with the ordering by selection method does not depend on the original ordering of the list. For a list that is already almost correct at the beginning, this method will be, therefore, considerably slower than the previous one. On the other hand, ordering by selection sort has the advantage of being more balanced. It means that the time it takes to sort a messy list is slightly longer than that of sorting an already sorted one. The other significant advantage is to minimize data movement: whenever any two elements are moved to the end, one of them will be in its final position and, therefore, will not be removed anymore.

In the following table, the statistical analysis of the performance of the algorithm in sorting a list of N elements is reported.

	Minimum	Average	Maximum
Comparisons	$\frac{N^2 - N}{2}$	$\frac{N^2 + N}{2}$	$\frac{N^2 + N}{2}$
Displacements	$3(N - 1)$	$N(\log N + 0.57)$	$\frac{N^2}{4} + 3(N - 1)$

THE SHELL SORTING

We have seen that the behaviour of the insertion and selection sort algorithms somewhat goes the opposite. The selection method moves items efficiently, but it requires many comparisons. On the other hand, the insertion method makes fewer comparisons but is inefficient at moving items. We now want to look for another algorithm that is affected as little as possible by the same problems as the other two.

If we analyse the insertion algorithm, we notice that it can move elements of one position at a time because it only compares contiguous items. Therefore, we could sort elements that are far apart if we modify this method first to compare elements that are not contiguous. Then, only the closest elements would be sorted. Finally, the difference between the elements to be compared would be reduced to one to ensure that the list is completely sorted. This sorting algorithm (also called increment-decreased sorting) was invented in 1959 by D.L. Shell, from which the name of this sorting method derives. Before we describe the algorithm in detail, let us explain it with a simple example.

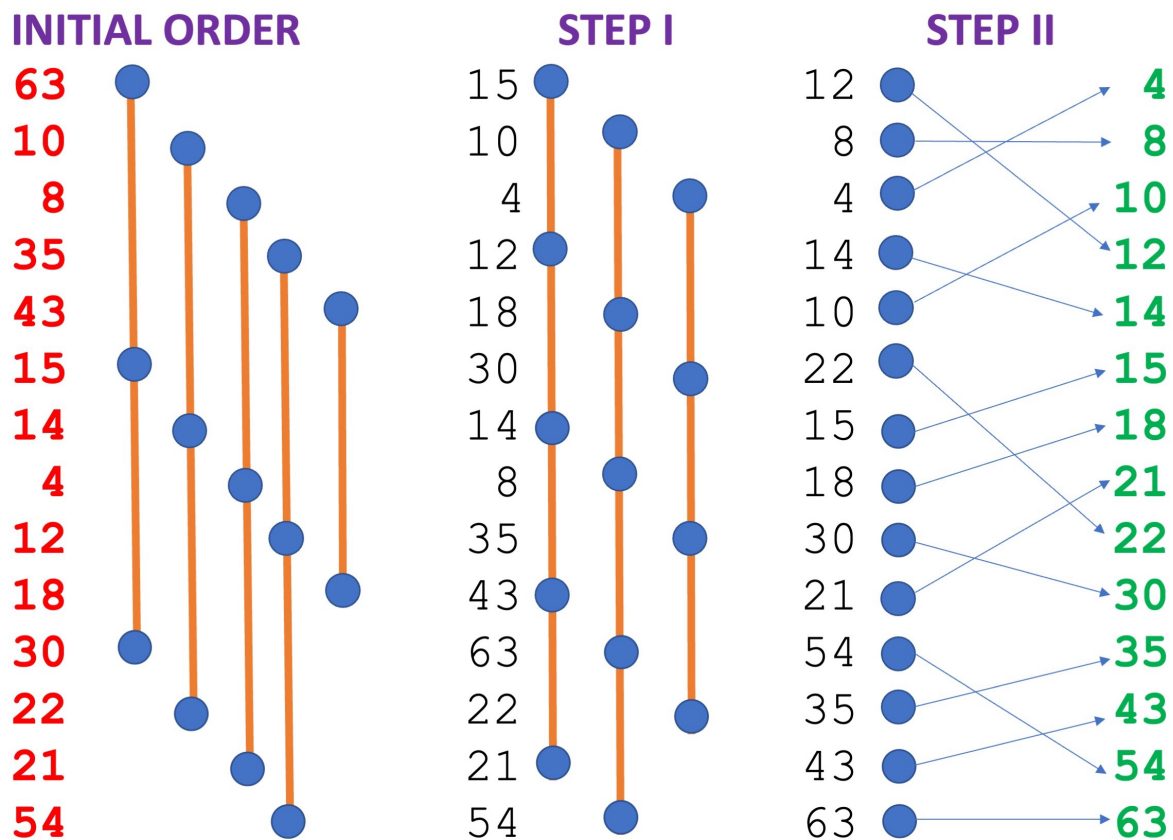


Figure 3: This example shows sorting a set of cards using the shell algorithm. In the first two steps, the dots connected with lines indicated the subset of numbers that are ordered. The last step is performed using the insertion method.

The left side of the Figure 3 shows a list of unsorted numbers. We start sorting all the numbers located five places apart in the initial step. In the next step, numbers only three places away are considered. Finally, in Step II, the comparison of contiguous numbers using insertion sort is used to complete the ordering. We notice that even if we take three only steps through all the numbers, the initial steps first move the numbers very close to their final position so that in the last step (when the comparison is made only between contiguous elements), all elements are already very close to their final sorted position making the insertion sorting very fast.

It is not necessary to choose the sequence of increments 5, 3, and 1 as in the given example; other choices may be just as good or even better. Although many studies have been done on this algorithm, no one has yet been able to prove that one choice of for the set of increments is better than another. If the increments are chosen very close together, as we did in the example, it will be necessary to make more moves, but each one will probably be faster than the previous one. If the increments decrease rapidly, it will take fewer moves, but they will be longer. The only essential rule is that the final increment equals one to obtain the wholly sorted list during the last check.

The program is an example of implementation of the Shell sorting algorithm. The sorting is performed by calling the C++ function **Shell_sort()**.

```
#include<iostream>
using namespace std;
```

```

/*
 * Shell sorting function
 */

void shell_sort (int *list, int n) {
    int h, i, j, t;
    for (h = n; h /= 2;) {
        for (i = h; i < n; i++) {
            t = list[i];
            for (j = i; j >= h && t < list[j - h]; j -= h) {
                list[j] = list[j - h];
            }
            list[j] = t;
        }
    }
}

/*
 * Testing program
 */

int main () {
    int list[] = {4, 65, 2, -31, 0, 99, 2, 83, 782, 1};
    int n = sizeof list / sizeof list[0];
    int i;
    for (i = 0; i < n; i++)
        cout << list[i] << " ";
    cout << "\n";
    shell_sort(list, n);
    for (i = 0; i < n; i++)
        cout << list[i] << " ";
    cout << "\n";
    return 0;
}

```

The statistical analysis of the Shell sort algorithm performance is very complicated: the number of comparisons and displacements can be estimated only with special conditions. However, it would be helpful to know how these numbers depend on the choice of increments to make the best choice. However, even without rigorous analysis, we can run some tests with a calculator to see that this method is better than the previous two.

For example, shell sorting studies have found that the number of moves to sort a list of N elements is between $N^{1.25}$ and $1.6N^{1.25}$, which is a substantial improvement over insertion sort.

SOME FINAL CONSIDERATIONS

In studying sorting algorithms, we considered having to sort lists of numbers. We used this limitation because we were interested in studying the different algorithms without worrying about the data we were working on. In practice, however, ordering a list of numbers is only a particular case; more generally, the sorting algorithms have as initial data a list of elements composed of two fundamental parts: the key part and the argument part. Let us clarify this concept with an example. Suppose we have a list in which the single element consists of a person's surname, first name, street, and telephone number. We may want to sort the items in this list only by the last name and not by the item's overall value. In this case, we call the part of the element consisting only of the surname key and the rest argument. Studying the

sorting algorithms, we have seen that the two parameters on which their efficiency depends the most are the number of comparisons and the number of exchanges between the elements to be sorted.

On the other hand, the speed of these two operations depends on the length of the elements: the comparison operation is performed only on the element's key, while the exchange operation is on the whole element.

SUMMARY

We can summarize the difference in the methods presented as follows.

1. There is a clear division between the natural methods, i.e., insertion and selection (N^2) sorting, and the advanced methods Shell ($N^{1.25}$).
2. The insertion method always represents the best choice in the case of an already sorted list.
3. The selection method, usually less efficient than the insertion method, is better in the case of elements consisting of key and argument.
4. The Shell method represents a significant improvement over insertion and selection.